

Interactive Computer Graphics Final Project

Real-time Smoke Simulation and Shading

蔡平樂 b11201024

許佑威 b11901089

June 7, 2026

1 Introduction

This project implements a real-time volumetric smoke simulation that runs entirely in the browser using WebGL. We start from the open-source 2D fluid simulation by PavelDoGreat [1] and extend it in several directions:

1. The 2D solver is lifted into a full 3D solver.
2. A temperature field drives a buoyancy force, and vorticity confinement restores small-scale swirling detail lost to numerical dissipation.
3. The volume is displayed by a per-pixel ray march with soft shadows.
4. A depth pre-pass records the eye-distance of solid surfaces so the ray march terminates correctly at opaque boundaries.
5. A first-person camera and a projectile system let the user explore the scene and trigger smoke bursts on impact.

All simulation passes run as GLSL fragment shaders on a full-screen quad, reading from and writing to floating-point FBOs (framebuffer objects).

2 Physics Simulation

The physics simulation is based on the original 2D solver by PavelDoGreat [1], adding a third dimension and extra fields for temperature and buoyancy.

2.1 Per-Frame Solver Steps

`step(dt)` in `simulation.js` executes the following passes each frame.

Buoyancy. Hot smoke rises and dense smoke sinks (`passes/buoyancy.js`):

$$v_y += \Delta t (B \cdot T - W \cdot |\rho|), \quad B = 0.1, \quad W = 0.001$$

Vorticity confinement. The curl $\boldsymbol{\omega} = \nabla \times \mathbf{v}$ is computed with six-neighbour finite differences, and a restoring force is injected:

$$\mathbf{f} = \varepsilon (\hat{\boldsymbol{\eta}} \times \boldsymbol{\omega}), \quad \hat{\boldsymbol{\eta}} = \text{normalize}(\nabla|\boldsymbol{\omega}|), \quad \varepsilon = 30$$

Pressure projection. Incompressibility is enforced in three sub-steps:
(i) divergence $\nabla \cdot \mathbf{v}$ with six-neighbour differences and no-slip boundaries;
(ii) 25 Jacobi iterations warm-started at $0.8\times$ the previous pressure:

$$p = \frac{1}{6}(p_L + p_R + p_B + p_T + p_F + p_K - \text{div});$$

(iii) velocity correction $\mathbf{v} \leftarrow \mathbf{v} - \frac{1}{2}\nabla p$.

Semi-Lagrangian advection. Velocity, density, and temperature are each back-traced and resampled (`passes/advection.js`):

$$\mathbf{uvw}_{\text{prev}} = \mathbf{uvw} - \Delta t \cdot \mathbf{v}(\mathbf{uvw}), \quad q^{n+1} = q^n(\mathbf{uvw}_{\text{prev}}) / (1 + d \Delta t)$$

2.2 Smoke Emitters

Smoke is injected by Gaussian splats (`passes/splat.js`): $q(\mathbf{uvw}) += \mathbf{c} \cdot \exp(-|\mathbf{uvw} - \mathbf{p}|^2/r)$. Each emitter is managed by an `EmitterScheduler` (`passes/emitter.js`) supporting a `startTime/endTime` window, an optional per-frame *trajectory callback* that overrides position or intensity, and an initial *burst* ($4\times$ radius, $2\times$ temperature) that immediately fills the local neighbourhood.

3 Adding 3D Models

`createSceneGeometry()` assembles the static scene from box primitives:

Object	Size (W×H×D)	Colour (RGB)
Floor	$16 \times 0.1 \times 16$	Dark grey (0.18, 0.20, 0.24)
Red box	$2 \times 2 \times 2$	Brick red (0.84, 0.35, 0.22)
Teal box	$1.5 \times 1.5 \times 1.5$	Teal (0.28, 0.78, 0.82)
Yellow box	$1 \times 1 \times 1$	Sand (0.75, 0.72, 0.36)
Front/back walls	$8 \times 4 \times 0.2$	Slate (0.32, 0.35, 0.38)
Left/right walls	$0.2 \times 4 \times 16$	Slate (0.32, 0.35, 0.38)

Each box also registers an AABB entry in `_colliders` for projectile collision detection (Section 6).

4 Shading

4.1 3D Model Shading: Blinn-Phong

Solid geometry is rendered with the `_MODEL_FRAG` shader in `model.js`. The key-light direction is fixed at $(0.4, 0.8, 0.45)$ (normalised in-shader) and the eye position is supplied per-frame via the `uEye` uniform:

$$I = \mathbf{c}_{\text{base}} \underbrace{(\max(\mathbf{N} \cdot \mathbf{L}, 0) \cdot 0.7 + 0.3)}_{\text{diffuse + ambient}} + \underbrace{\max(\mathbf{N} \cdot \mathbf{H}, 0)^{48} \cdot 0.4}_{\text{specular}}$$

where $\mathbf{H} = \text{normalize}(\mathbf{L} + \mathbf{V})$. The constant 0.3 provides a soft ambient floor; the exponent 48 gives a moderately tight specular highlight. Normals are brought to world space using the upper-left 3×3 submatrix of the model matrix.

4.2 Smoke Shading: Volumetric Ray Marching

The smoke volume is rendered by `drawRayMarch` in `passes/render.js`.

Camera and ray construction. An FPS camera basis $(\hat{F}, \hat{R}, \hat{U})$ is built from yaw and pitch angles at runtime:

$$\hat{d} = \text{normalize}\left(\hat{F} + \text{ndc}_x \cdot \text{aspect} \cdot \tan \frac{\text{FOV}}{2} \cdot \hat{R} + \text{ndc}_y \cdot \tan \frac{\text{FOV}}{2} \cdot \hat{U}\right)$$

Field of view is 60° ; the aspect ratio is computed from the canvas each frame.

Ray-box intersection. The ray is intersected with the volume AABB (half-extent 5.0, centre $(0, -2, 12)$) using the slab method, yielding t_{start} and t_{end} .

Step-and-accumulate loop. The interval $[t_{\text{start}}, t_{\text{end}}]$ is divided into 64 equal steps (`MAX_STEPS`). At each sample position $\mathbf{p}$:

1. The volume UVW is $(\mathbf{p} - \mathbf{c}_{\text{vol}})/(2r) + 0.5$. Samples with density $\rho < 0.001$ are skipped.
2. **Soft shadow.** Three steps of length 0.12 are marched toward the light; accumulated density attenuates illumination: $\ell = \exp(-\sigma_{\text{sh}} \cdot \text{DENSITY_SCALE} \cdot \text{ABSORPTION} \cdot \text{SHADOW_STEP})$.
3. **Temperature tint.** Sample temperature interpolates between cool blue-white $(0.85, 0.90, 1.00)$ and warm orange $(1.00, 0.55, 0.10)$: $\mathbf{c}_{\text{tint}} = \text{mix}(\text{cool}, \text{warm}, \text{clamp}(T \cdot 0.5, 0, 1))$.

4. Front-to-back compositing:

$$\alpha = 1 - e^{-\rho \Delta s \cdot \text{ABS}}, \quad \mathbf{c}_{\text{acc}} += \tau \alpha \mathbf{c}_{\text{tint}}(\mathbf{c}_{\text{light}} \ell + 0.35), \quad \tau *= (1 - \alpha)$$

Marching stops early when $\tau < 0.01$.

The 0.35 ambient term prevents smoke in full shadow from being completely invisible. The result is blended over the background using premultiplied alpha (`blendFunc(ONE, ONE_MINUS_SRC_ALPHA)`).

5 Depth Buffer Integration

5.1 Depth Pre-Pass

Before ray marching, `drawModelDepthCapture()` renders all scene primitives into an off-screen RGBA half-float FBO (`_modelScreenFBO`) with an attached 16-bit depth renderbuffer. The shader (`_MODEL_DEPTH_FRAG`) writes the eye-to-surface Euclidean distance into the *alpha* channel:

$$\text{gl_FragColor} = \text{vec4}(0, 0, 0, |\mathbf{w} - \mathbf{eye}|)$$

Alpha zero signals no solid surface at that pixel. The pass enables `DEPTH_TEST (LEQUAL)`, depth writes, and back-face culling, using the same 60° perspective and view matrix as the ray march so the recorded distances are consistent.

5.2 Ray-March Integration

Inside `drawRayMarch`, the pre-pass texture is bound as `uModelBuffer`. For each pixel, the alpha value gives the model depth d_{model} , which truncates the march interval:

$$t_{\text{end}} = \min(t_{\text{AABB}}, d_{\text{model}})$$

When alpha is zero, d_{model} is set to 10^9 so the ray travels the full AABB extent. If $t_{\text{end}} \leq t_{\text{start}}$, the fragment outputs transparent black immediately, saving the cost of the loop entirely.

5.3 Render Order

The final frame is assembled in three ordered draw calls:

1. Depth pre-pass → off-screen FBO.
2. Rasterise solid geometry → default framebuffer (Blinn-Phong, depth test on).
3. Ray march → default framebuffer (reads pre-pass depth, blended over rasterised models with `blendFunc(ONE, ONE_MINUS_SRC_ALPHA)`).

6 Interactive Controls

6.1 First-Person Camera

Camera control in `input.js` uses the Pointer Lock API for raw mouse deltas. Movement basis is derived from the current yaw so that WASD stays in the XZ plane regardless of pitch.

Key / Input	Action
W/S	Move forward / backward
A/D	Strafe left / right
Shift/Ctrl	Move up / down
Mouse move (locked)	Rotate yaw and pitch
1–4	Select smoke-grenade colour (hotbar)
Left click	Throw the selected smoke grenade
P	Pause / resume simulation
Space	Toggle depth visualisation

Pitch is clamped to $\pm 0.44\pi$. Move speed is 0.05 world units per key event; mouse sensitivity is 0.002 rad/pixel; both are configurable in `config.js`.

6.2 Projectile System: Smoke Grenades

Left-clicking (after pointer lock) throws a *smoke grenade*—an instance of a GLB mesh (`assets/models/ball.glb`) loaded once at start-up by `loadProjectileTemplate` and cloned per throw by `createProjectileModel`. Each grenade spawns 0.6 units ahead of the camera with initial velocity $\mathbf{v}_0 = 8.0 \hat{F} + (0, 1.2, 0)$; the upward bias gives it a natural arc.

Hotbar and grenade types. `PROJECTILE_TYPES` in `model.js` defines four grenade colours (white, red, blue, green), each pairing a model tint with a matching smoke colour. Pressing 1–4 calls `setProjectileType`, which updates `currentProjectileType` and refreshes an on-screen hotbar (`updateProjectileHUD`): the selected slot is highlighted and a `#weapon-name` label shows the grenade’s display name (e.g. “Red smoke grenade”). Cloned primitives reuse the template’s geometry buffers and simply override `baseColor` with the type’s tint, avoiding redundant GPU uploads.

Impact response. Every frame, `updateProjectiles(dt)` integrates gravity ($a_y = -3.0$) and tests the motion segment against each AABB in

`_colliders` using the slab method. On the nearest hit, `onProjectileHit` converts the impact point to volume UVW and performs three Gaussian splats (`splat3D`, radius 0.006):

1. the grenade’s smoke colour into the *density* field, producing a coloured smoke burst;
2. a uniform heat value (0.35, 0.35, 0.35) into the *temperature* field, so the burst rises by buoyancy;
3. the normalised impact velocity, scaled by 0.4, into the *velocity* field, so the burst inherits the grenade’s direction of travel as an initial impulse.

Projectiles surviving more than 5 seconds are removed automatically. Because projectiles participate in the depth pre-pass, they correctly occlude the smoke volume.

6.3 Depth Visualisation

Pressing `Space` toggles the flag `config.SHOW_DEPTH_VIZ`. In this mode, `drawDepthViz` reads the alpha channel of the pre-pass FBO and maps eye-distance to greyscale ($1 - d/20$), giving a live view of the depth buffer for debugging.

7 Limitations and Future Work

Smoke does not flow around solids. The depth pre-pass makes smoke *appear* to stop at surfaces, but the fluid solver has no solid boundary conditions. Velocity and density fields pass through geometry unimpeded. A proper fix would stamp a solid-occupancy mask each frame and enforce no-slip conditions in the divergence and pressure-gradient passes.

Volume resolution. At 64^3 voxels, fine detail is lost at close range due to trilinear smearing. Doubling the linear resolution to 128^3 would require a 1024×1024 atlas ($4\times$ the memory and roughly $8\times$ the shader work), making higher resolution expensive without further optimisation such as adaptive grid refinement.

Shadow quality. Only three fixed-length shadow-ray steps are used, which underestimates occlusion inside thick smoke. More steps, or an adaptive step size based on local density, would improve accuracy at a proportional GPU cost.

8 References

References

- [1] P. Dobrev (PavelDoGreat), *WebGL Fluid Simulation*.
<https://github.com/PavelDoGreat/WebGL-Fluid-Simulation>
- [2] A. Chan, *Real-time Fire Simulation*.
<https://andrewkchan.dev/posts/fire.html>